ACID Properties

Module Title and Code: Introduction to Databases, UFCFTK-30-1

Submitted By: Raunak Kumar Singh

Student Number: 24071246

Orbund Id: 11281

Submitted To: Rabin Gautam

TASK 4 (20%)

What are ACID properties in the context of database transactions? Explain with 1 example.

Answer:

ACID properties are a set of rules that make sure database transactions are processed correctly and data remains reliable. A transaction is a group of operations that are treated as a single unit. **ACID** stands for **Atomicity**, **Consistency**, **Isolation**, and **Durability**, and each properly helps maintain data integrity in a database system.

A- <u>Atomicity</u>

Atomicity means that a transaction must either complete fully or not happen at all. If any part of the transaction fails, all changes made during that transaction are reversed.

For example, in an online shopping system when a customer places an order:

```
BEGIN TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE product_id = 101;
INSERT INTO orders (product_id, user_id) VALUES (101, 1);
COMMIT;
```

```
Connect to MSSQL
BEGIN TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE product_id = 101;
INSERT INTO orders (product_id, user_id) VALUES (101, 1);
COMMIT;
```

If the system crashes after reducing the stock but before creating the order, the transaction will be rolled back. This ensures that the stock is not reduced without recording the order.

C-Consistency

Consistency ensures that the database always remains in a valid state. All rules, such as constraints and relationships, must be followed before and after a transaction.

For example:

```
BEGIN TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE product_id = 101;
COMMIT;
```

```
Connect to MSSQL
BEGIN TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE product_id = 101;
COMMIT;
```

If the stock is already zero, the update should not be allowed. This prevents invalid data such as negative stock and keeps the data accurate.

I-Isolation

Isolation means that multiple transactions running at the same time should not interfere with each other.

For example, if two users try to buy the last available product at the same time:

Transaction 1

BEGIN TRANSACTION;

UPDATE products SET stock = stock - 1 WHERE product_id = 101;

COMMIT;

Transaction 2

BEGIN TRANSACTION;

UPDATE products SET stock = stock - 1 WHERE product_id = 101;

COMMIT;

```
Connect to MSSQL
-- Transaction 1
BEGIN TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE product_id = 101;
COMMIT;

-- Transaction 2
BEGIN TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE product_id = 101;
COMMIT;
```

Only one transaction will succeed, while the other will wait or fail. This prevents issues like overselling the same product.

D-Durability

Durability ensures that once a transaction is completed, the changes are permanently saved in the database.

For example:

BEGIN TRANSACTION;

INSERT INTO orders (product_id, user_id) VALUES (101, 1);

COMMIT;

```
Connect to MSSQL
BEGIN TRANSACTION;
INSERT INTO orders (product_id, user_id) VALUES (101, 1);
COMMIT;
```

After the transaction is committed, the order will remain saved even if the system crashes immediately afterward. This ensures that important data is not lost.